

Case Swarm - Data Flows Explained

First-time guide: Salesforce, Azure, and Microsoft Teams

Updated 2026-07-24 | Live path: Azure bridge + SignalR (Option D)

1. Big picture

Case Swarm has two jobs: (1) Swarming - create a Teams Team or group Chat for a Case, add SMEs, install the bot, show a Case Adaptive Card. (2) Live chatting - let a Salesforce agent talk in that Teams chat from the Case page.

Three applications always involved:

App	Owns
Salesforce	Case, agent UI (LWC), Apex, Adaptive Card REST
Azure	Relay Functions + SignalR (dumb plumbing)
Microsoft 365	Teams chat, Graph, Bot Framework

Salesforce <-> Azure (relay / SignalR) <-> Teams / Graph / Bot
SF decides WHAT. Azure moves bytes. Microsoft is the real chat room.

2. Two Microsoft doors

Do not conflate Graph and Bot Framework:

Door	Used for
Microsoft Graph (MS_Graph / GRAPH_*)	Create chat/team, members, install app, list history
Bot Framework (MS_Bot_Framework / MICROSOFT _APP_*)	Post as Case Swarm bot; receive Teams activities

Graph app-only cannot reliably post live chat messages as this bot needs. Outbound live text and Adaptive Cards use Bot Framework instead.

3. What is a callout?

Term	Meaning	SF callout limit?
Apex callout	Apex Http.send / Named Credential on SF server	Yes
Browser fetch	LWC fetch() from agent browser to Azure	No
Azure HTTPS	Function calls Graph or Bot Framework	No (not SF)

Live outbound send (Azure bridge) - production:

Browser LWC --fetch--> Azure chatSend --HTTPS--> Bot Framework --> Teams
(not Apex) (not Salesforce)

Apex fallback send (bridge off): LWC -> Apex sendMessage -> MS_Bot_Framework callout -> Bot -> Teams.

4. Flow A - Start a Chat swarm

Agent starts Quick Chat -> SF asks Graph to create chat -> Case Active -> bot installed -> Teams tells Azure bot joined -> Azure tells SF -> SF stores serviceUrl and posts Case card.

Step	App	What happens
1	SF	caseSwarm LWC: pick SMEs, start Chat swarm
2-3	SF->Graph	Apex callout MS_Graph: create chat + members (Azure_AD_Email_Id__c)
4-5	SF	Case: Active, Type=Chat, Team_Id=chatId, Team_Url
6	SF->Graph	Queueable installs Teams app (RSC for no-@mention read)
7-8	Teams->Azure->SF	conversationUpdate via relay; Apex REST stores Swarm_Bot_Service_Url__c
9	SF->Bot	Queueable posts Case Adaptive Card as bot

Code: TeamsSwarmController, TeamsSwarmService, TeamsSwarmBotInstallQueueable, teamsBotRelay.js, TeamsBotMessagingResource, TeamsBotInstalledCardQueueable, TeamsBotConversationService.

5. Flow B - Team swarm (short)

Graph creates M365 group then enables Team (async retry possible). Members added. Open in Teams only today - live SF text bridge and SignalR are Chat-swarm only.

6. Flow C - Open live chat in SF

Flags: Use_Azure_Chat_Bridge__c + Use_Azure_SignalR__c on Teams_Bot_Config CMDT.

Step	Mechanism	What happens
1	Apex session	getChatBridgeSession: HMAC, chatId, serviceUrl, paths
2-3	fetch + Graph	GET /api/chatHistory one-shot via Azure
4-7	fetch + WSS	negotiate, WebSocket, joinChat (group=chatId)
8	UI	Live: SignalR push - no continuous Graph poll while connected

Code: caseSwarmChatCore.js, caseSwarmChatSignalR.js, chatHistory.js, negotiate.js, joinChat.js.

7. Flow D - Inbound Teams -> Salesforce

SME types in Teams -> Bot Framework -> Azure relay -> SignalR newMessage -> LWC appends bubble. No Graph per message while SignalR is up. No Swarm_Message__c write on live Azure path.

Teams user -> Bot Framework -> teamsBotRelay -> SignalR group=chatId
-> LWC appendPushedMessage (dedupe)

Card Edit/Save invokes still go Relay -> Salesforce Apex REST (business logic in Apex).

8. Flow E - Outbound SF -> Teams (no Graph)

Agent types in SF -> browser POST Azure /api/chatSend -> Azure posts Bot Framework using Case Swarm_Bot_Service_Url__c + Swarm_Team_Id__c -> Teams shows Case Swarm bot -> SignalR echoes to LWC. Graph is not called.

Step	Where	SF Apex callout?
1	LWC handleSend / sendViaAzure	No
2	Browser fetch POST /api/chatSend + HMAC	No
3	chatSend.js -> sendAgentMessage	No
4	botSend.js POST .../v3/conversations/{id}/activities	No
5	SignalR outbound echo	No

Exact files: caseSwarmChatCore.js (sendViaAzure), relay/src/functions/chatSend.js, relay/src/botSend.js (postActivity).
Apex fallback: TeamsSwarmChatController.sendMessage -> TeamsBotConversationService.sendAgentText (THIS is an SF callout via MS_Bot_Framework).

9. Why the Azure relay exists

Salesforce Apex REST treats Authorization as a Salesforce session. Bot Framework always sends its JWT in Authorization. Relay authenticates to Salesforce (Connected App), puts SF token in Authorization, and moves Bot JWT to X-Bot-Framework-Authorization for TeamsBotJwtValidator.

Bot Framework	Azure Relay	Salesforce
Authorization: Bot JWT	-> Authorization: SF token	
	X-Bot-Framework-Authorization: Bot JWT	

10. Cheat sheet - who talks to whom

Action	SF callout?	Browser fetch?	Graph?	Bot Framework?	SignalR?
Create Chat swarm	Yes Graph	No	Yes	Card later	No
Open history once	No	Yes Azure	Via Azure	No	No
SignalR connect	No	Yes	No	No	Yes
Inbound Teams text	No	Push in	No	Receive relay	Push
Outbound (bridge)	No	Yes chatSend	No	Yes	Echo
Outbound (Apex)	Yes Bot NC	No	No	Yes	No
Card Edit invoke	Relay->Apex	No	No	Invoke	No

11. One-page memory aid

SWARM: SF Apex --Graph--> Teams chat --> bot install
Teams --Bot--> Azure relay --Apex REST--> serviceUrl + card

LIVE OPEN: SF LWC --fetch--> Azure history (Graph once) --> SignalR join

INBOUND: Teams --Bot--> Azure --SignalR--> SF LWC
(no Graph per message; no Apex per message)

OUTBOUND: SF LWC --fetch--> Azure chatSend --Bot Framework--> Teams
(+ SignalR echo). Not Graph. Not SF Apex callout on bridge path.

12. Related markdown in docs/

- Case-Swarm-Data-Flows-Explained.md - this guide (source)
- Case-Swarm-SignalR-Option-D-Guide.md - setup and troubleshooting
- Case-Swarm-Chat-Bridge-Options-and-Limits.md - options and limits
- Case-Swarm-Architecture-and-Data-Flows.md - deeper diagrams